

Instrucciones

- Lea con detenimiento y desarrolle **individualmente** cada una de las actividades a realizar durante la experiencia.
- Cree un archivo comprimido con extensión **.zip**, **.rar** o **.tar.gz** que incluya todos los archivos de su proyecto Qt utilizados para el desarrollo. El nombre del archivo debe tener el siguiente formato: **TEL102_C4_Nombre_Apellido.zip** (Ej. TEL102_C4_Patricio_Olivares.zip), sin incluir tildes.
- Enviar el archivo a través de la página de aula del ramo, sección “Control 4” hasta las 10:50:00 del día Martes 19/11/2024 hora local continental de Chile.
- Utilice herramientas conocidas o aprendidas en clases. **No copie literalmente de recursos online.**
- La evaluación es de carácter individual. Está **estrictamente prohibido** tanto la comunicación entre estudiantes, así como el uso de herramientas generativas tales como chatGPT, Github copilot, y similares.
- Sea riguroso con las instrucciones de desarrollo.
- ¡Éxito!

1. Gestión de Stock Mensual - StockMonitor Systems

La empresa **StockMonitor Systems** ha sido contratada por un centro logístico para desarrollar una herramienta que permita monitorear y ajustar los valores de stock mensual en tres meses: *Enero*, *Febrero*, y *Marzo*. Debido a la naturaleza cambiante de las operaciones, es fundamental que los operadores puedan modificar dinámicamente estos valores y visualizar los cambios en tiempo real.

La aplicación debe proporcionar una interfaz gráfica que incluya controles deslizantes (**QSlider**) para ajustar los valores de stock de cada mes. Estos ajustes se reflejarán instantáneamente en un gráfico de barras (**Bar Chart**) interactivo, donde cada barra representa el stock de un mes específico. El gráfico debe actualizarse automáticamente conforme se realicen cambios en los deslizadores.

Para desarrollar la aplicación, utilice el proyecto Qt disponible en la plataforma Aula del curso. Este proyecto contiene un elemento **QGraphicsView** modificado para que pueda asignársele un objeto **QChart** (revise el siguiente [enlace](#) para más detalles).

Además, puede ver un ejemplo de la interfaz y su funcionamiento final en el siguiente [enlace](#).

1. (10pts) Complete la interfaz de usuario utilizando Qt Designer. La interfaz debe contener tres controles deslizantes de tipo **QSlider** y un elemento **QChartView** para el gráfico de barras. Cada **QSlider** debe iniciar con un valor de 99, y su rango debe estar entre 0 y 99. El gráfico debe estar configurado para que el eje x represente los meses:

- El valor 1 corresponde a **Enero**.
 - El valor 2 corresponde a **Febrero**.
 - El valor 3 corresponde a **Marzo**.
2. (30pts) En la clase **Widget** de la ventana principal, agregue lo siguiente:
- Tres atributos privados para almacenar los valores del stock de los meses *Enero*, *Febrero* y *Marzo*.
 - Dos atributos privado de tipo **QBarSeries *** y **QBarSet ***, que almacenarían la información relativa al gráfico que se mostrará en la ventana.
 - Complete el constructor de la clase, inicializando los atributos y configurando el gráfico de barras.
3. (30pts) Cree e implemente los siguientes métodos en la clase **Widget**:
- Un método privado **void updatePlot()** que se encargue de actualizar el gráfico cada vez que sea llamado.
 - Un método slot público **void updateEnero(int val)**, que reciba como parámetro el valor asociado al stock del mes de enero, actualice el atributo correspondiente y llame al método **updatePlot()** para reflejar el cambio en el gráfico.
 - Un método slot público **void updateFebrero(int val)**, que reciba como parámetro el valor asociado al stock del mes de febrero, actualice el atributo correspondiente y llame al método **updatePlot()** para reflejar el cambio en el gráfico.
 - Un método slot público **void updateMarzo(int val)**, que reciba como parámetro el valor asociado al stock del mes de marzo, actualice el atributo correspondiente y llame al método **updatePlot()** para reflejar el cambio en el gráfico.
4. (30pts) Configure las conexiones necesarias entre signals y slots para que el movimiento de los controles deslizantes (**QSlider**) modifique el gráfico en tiempo real. Asegúrese también de que toda memoria dinámica sea liberada adecuadamente en el destructor de la clase **Widget**.